DATABASE MANAGEMENT SYSTEMS (DBMS) — COMPLETE TVET NOTES

Course Outline

Introduction to Databases and DBMS

Database Concepts and Terminology

Database Models

Relational Database Model

Tables, Records and Fields

Keys and Constraints

Entity-Relationship Diagrams (ERD)

Database Normalization

SQL Introduction

SQL Data Definition Language (DDL)

SQL Data Manipulation Language (DML)

SQL Queries and Functions

Database Security, Backup and Recovery

Database Projects and Revision

TOPIC 1: INTRODUCTION TO DATABASES AND DBMS

1.1 What is Data?

Data refers to raw facts and figures that have not yet been processed.

Examples:

Joseph

20

85

ICT

These individual values are data.

1.2 What is Information?

Information is processed data that has meaning and can be used for decision-making.

Example:

Student: Joseph

Age: 20

Course: ICT

Average Mark: 85%

Grade: A

This is meaningful information.

1.3 Data vs Information

Data	Information
Raw facts	Processed data

May have little meaning by itself Has meaning

Used as input Used as output

Example: 85 Example: Joseph scored 85%

1.4 What is a Database?

A database is an organized collection of related data that can be stored, accessed, managed and updated electronically.

Example of a student database:

Student ID	Name	Course	Marks
101	Joseph	ICT	85
102	Joel	ICT	78
103	Mercy	ICT	90

1.5 What is a DBMS?

A Database Management System (DBMS) is software used to create, store, organize, manage, retrieve and manipulate data in databases.

Examples include:

PostgreSQL

MySQL

Microsoft SQL Server

Oracle Database

SQLite

Microsoft Access

MariaDB

1.6 Functions of a DBMS

A DBMS can:

Create databases

Create tables

Insert data

Update data

Delete data

Retrieve data

Control access

Provide backup and recovery

Maintain data integrity

Manage transactions

1.7 Advantages of DBMS

Reduces data duplication

Improves data security

Improves data consistency

Allows data sharing

Provides backup and recovery

Supports multiple users

Makes data retrieval easier

Maintains data integrity

1.8 Examples of DBMS

DBMS	Common Use
PostgreSQL	Web and enterprise applications
MySQL	Web applications
SQL Server	Business applications
Oracle	Enterprise systems
SQLite	Lightweight applications
MS Access	Small database applications

Revision Questions

Define data.

Define information.

Define a database.

What is a DBMS?

State five functions of a DBMS.

Give five examples of DBMS software.

State five advantages of using a DBMS.

TOPIC 2: DATABASE CONCEPTS AND TERMINOLOGY

2.1 Database

A database stores related information in an organized manner.

2.2 Table

A table stores related data in rows and columns.

Example:

ID	Name	Age
1	Joseph	20
2	Joel	21

2.3 Record

A record is a complete row of information about one entity.

Example:

1 | Joseph | 20

This represents one student record.

2.4 Field

A field is a single data attribute or column.

Example:

ID

Name

Age

are fields.

2.5 Attribute

An attribute describes a characteristic of an entity.

For a student:

Student

|

├── StudentID

├── Name

├── Age

├── Course

└── Phone

2.6 Entity

An entity is a person, place, object, event or concept about which data is stored.

Examples:

Student

Employee

Product

Customer

Course

Department

2.7 Database Schema

A database schema describes the structure of a database.

It defines:

Tables

Columns

Data types

Relationships

Constraints

2.8 Database Instance

A database instance is the actual data stored in the database at a particular point in time.

2.9 Metadata

Metadata is data about data.

For example, information describing:

Table names

Column names

Data types

Constraints

Revision Questions

Define a table.

Define a record.

Define a field.

What is an entity?

What is an attribute?

Explain database schema.

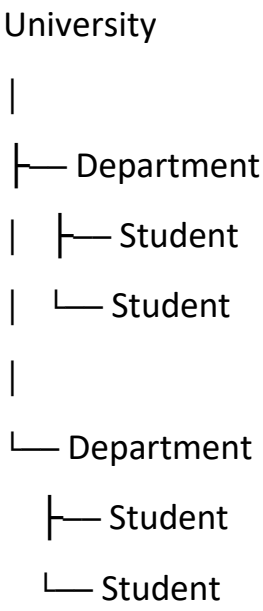
Differentiate between a field and a record.

TOPIC 3: DATABASE MODELS

A database model describes how data is organized and how relationships between data are represented.

3.1 Hierarchical Database Model

Data is organized in a tree structure.



Each child generally has one parent.

3.2 Network Database Model

Data is organized as records connected through relationships.

Unlike a strict tree, a record can participate in multiple relationships.

3.3 Relational Database Model

Data is stored in tables consisting of rows and columns.

Example:

Students

ID	Name	Course
1	Joseph	ICT
2	Mercy	ICT

This is the most commonly taught model for SQL-based relational databases.

3.4 Object-Oriented Database Model

Data is represented using objects and classes.

It is useful where applications work heavily with complex objects.

3.5 NoSQL Databases

NoSQL databases use models other than traditional relational tables.

Examples include:

Document databases

Key-value databases

Graph databases

Wide-column databases

An example document might look conceptually like:

```
{
  name: "Joseph",
  course: "ICT",
  marks: 85
}
```

3.6 Comparison

Model	Structure
Hierarchical	Tree
Network	Connected records
Relational	Tables
Object-oriented	Objects
NoSQL	Various non-relational structures

Revision Questions

What is a database model?

Explain the hierarchical model.

Explain the relational model.

State two characteristics of the relational model.

What is a NoSQL database?

Differentiate between relational and hierarchical databases.

TOPIC 4: RELATIONAL DATABASE MODEL

4.1 What is a Relational Database?

A relational database stores data in related tables.

Example:

Students

StudentID	Name	CourseID
1	Joseph	101
2	Joel	102

Courses

CourseID	CourseName
101	ICT
102	Business

The CourseID connects the two tables.

4.2 Relational Database Terminology

Relation

A table.

Tuple

A row in a relational table.

Attribute

A column.

Domain

The set of valid values that an attribute can contain.

4.3 Relationships

Tables can be related using keys.

Common relationships include:

One-to-One

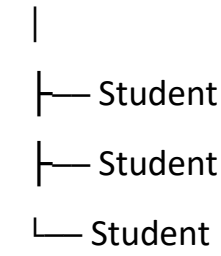
One record relates to one record.

Person —— Passport

One-to-Many

One record relates to many records.

Department



Many-to-Many

Many records relate to many records.

Student ↔ Course

This is normally implemented using a junction/associative table.

4.4 Referential Integrity

Referential integrity ensures that relationships between tables remain valid.

For example, a foreign key should generally refer to an existing primary-key value in the referenced table, unless the relationship allows NULL.

Revision Questions

Define a relational database.

What is a tuple?

What is an attribute?

Explain one-to-one relationships.

Explain one-to-many relationships.

Explain many-to-many relationships.

What is referential integrity?

TOPIC 5: TABLES, RECORDS AND FIELDS

5.1 Creating a Table

Example:

```
CREATE TABLE students (
   student_id INT,
   name VARCHAR(100),
   age INT,
   course VARCHAR(100)
);
```

5.2 Data Types

Common SQL data types include:

Data Type Purpose

INT Whole numbers

DECIMAL Exact decimal numbers

VARCHAR Variable-length text

CHAR Fixed-length text

DATE Dates

TIME Time values

BOOLEAN True/false values

Exact data types and features vary between DBMS products.

5.3 Inserting Records

```
INSERT INTO students
(student_id, name, age, course)
VALUES
(1, 'Joseph', 20, 'ICT');
```

Multiple records:

```
INSERT INTO students
(student_id, name, age, course)
VALUES
(2, 'Joel', 21, 'ICT'),
(3, 'Mercy', 19, 'Business');
```

5.4 Viewing Records

```
SELECT * FROM students;
```

5.5 Updating Records

```
UPDATE students
SET age = 22
WHERE student_id = 2;
```

5.6 Deleting Records

```
DELETE FROM students
WHERE student_id = 3;
```

Important: Always use a suitable WHERE condition when you only want to affect specific records.

Revision Questions

What is a table?

What is a record?

What is a field?

Write SQL to create a student table.

Write SQL to insert a student.

Write SQL to update a student.

Write SQL to delete a student.

TOPIC 6: KEYS AND CONSTRAINTS

Keys identify records and help establish relationships between tables.

6.1 Primary Key

A primary key uniquely identifies each record in a table.

Example:

```
CREATE TABLE students (
   student_id INT PRIMARY KEY,
   name VARCHAR(100),
   course VARCHAR(100)
);
```

Each student_id must be unique and cannot be NULL.

6.2 Foreign Key

A foreign key is a column that references a key in another table.

Example:

```
CREATE TABLE courses (
   course_id INT PRIMARY KEY,
   course_name VARCHAR(100)
);
```

Then:

```
CREATE TABLE students (
   student_id INT PRIMARY KEY,
   name VARCHAR(100),
```

```
    course_id INT,
    FOREIGN KEY (course_id)
        REFERENCES courses(course_id)
);
```

6.3 Candidate Key

A candidate key is a column or combination of columns that can uniquely identify a record and could potentially be selected as the primary key.

6.4 Composite Key

A composite key consists of two or more columns used together to uniquely identify a record.

Example:

```
PRIMARY KEY (student_id, course_id)
```

6.5 Alternate Key

A candidate key that was not selected as the primary key.

6.6 Constraints

Constraints are rules applied to database columns.

Common constraints:

PRIMARY KEY

Uniquely identifies records.

FOREIGN KEY

Maintains relationships.

NOT NULL

Prevents NULL values.

name VARCHAR(100) NOT NULL

UNIQUE

Prevents duplicate values.

email VARCHAR(150) UNIQUE

CHECK

Ensures a value satisfies a condition.

age INT CHECK (age >= 18)

DEFAULT

Provides a default value.

status VARCHAR(20) DEFAULT 'Active'

Revision Questions

Define a primary key.

Define a foreign key.

What is a composite key?

Explain candidate keys.

State five SQL constraints.

Differentiate between a primary key and foreign key.

TOPIC 7: ENTITY-RELATIONSHIP DIAGRAMS

7.1 What is an ERD?

An Entity-Relationship Diagram (ERD) is a visual representation of entities, attributes and relationships in a database.

7.2 Components of an ERD

Entity

Represents an object.

Examples:

STUDENT

COURSE

TEACHER

Attribute

Describes an entity.

STUDENT

├── StudentID

├── Name

├── Age

└── Phone

Relationship

Shows how entities are connected.

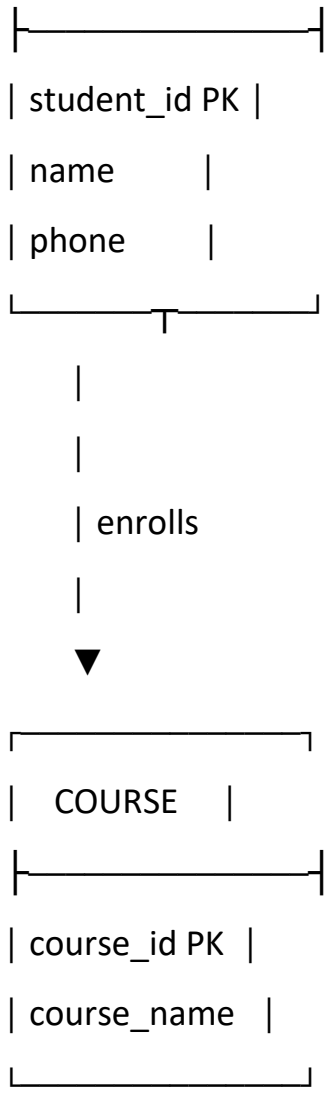
STUDENT ─── ENROLLS ─── COURSE

7.3 Example ERD

Suppose a college has students and courses.

┌──────────────┐

│ STUDENT │



For a many-to-many relationship, use a junction table:



▲

|

|

COURSE

7.4 Cardinality

Cardinality describes how many instances of one entity can be associated with another.

Examples:

1:1

1:M

M:N

Revision Questions

Define ERD.

State three components of an ERD.

Define an entity.

Define an attribute.

What is cardinality?

Explain 1:1, 1:M and M:N relationships.

Draw an ERD for a student and course database.

TOPIC 8: DATABASE NORMALIZATION

8.1 What is Normalization?

Normalization is the process of organizing data to reduce unnecessary duplication and improve data integrity.

8.2 Why Normalize Databases?

Normalization helps:

Reduce redundancy

Avoid update problems

Improve consistency

Improve database structure

Reduce unnecessary duplication

8.3 First Normal Form — 1NF

A table is in 1NF when each field contains a single, atomic value and repeating groups are removed.

Poor design

Student	Phone
Joseph	0711, 0722

Better:

Student	Phone
Joseph	0711

Joseph 0722

8.4 Second Normal Form — 2NF

A table should:

Be in 1NF.

Have no partial dependency of non-key attributes on part of a composite key.

This is particularly important when a table has a composite primary key.

8.5 Third Normal Form — 3NF

A table should:

Be in 2NF.

Have no inappropriate transitive dependency of non-key attributes on other non-key attributes.

8.6 Example

Poor design:

StudentID	StudentName	CourseID	CourseName
1	Joseph	101	ICT
2	Joel	101	ICT

Better design:

Students

StudentID StudentName

1 Joseph

2 Joel

Courses

CourseID CourseName

101 ICT

Enrollment

StudentID CourseID

1 101

2 101

Revision Questions

Define normalization.

Why is normalization important?

Explain 1NF.

Explain 2NF.

Explain 3NF.

State three advantages of normalization.

TOPIC 9: INTRODUCTION TO SQL

9.1 What is SQL?

SQL (Structured Query Language) is a language used to communicate with relational databases.

SQL can be used to:

Create databases and tables

Insert data

Retrieve data

Update data

Delete data

Control access

Manage transactions

9.2 SQL Categories

Important SQL categories include:

DDL — Data Definition Language

Used to define database structures.

Examples:

CREATE

ALTER

DROP

TRUNCATE

DML — Data Manipulation Language

Used to manipulate data.

Examples:

INSERT

UPDATE

DELETE

DQL — Data Query Language

Often used to refer to data retrieval using:

SELECT

DCL — Data Control Language

Used to control permissions.

Examples include:

GRANT

REVOKE

TCL — Transaction Control Language

Used to manage transactions.

Examples:

COMMIT

ROLLBACK

SAVEPOINT

9.3 SQL Syntax

Example:

```
SELECT name
FROM students;
```

SQL keywords are generally written in uppercase for readability, although many DBMSs do not require uppercase.

Revision Questions

Define SQL.

What does DDL stand for?

What does DML stand for?

What does DCL stand for?

What does TCL stand for?

Give three examples of DDL commands.

TOPIC 10: SQL DATA DEFINITION LANGUAGE

10.1 CREATE

Create a database:

```
CREATE DATABASE college;
```

Create a table:

```
CREATE TABLE students (
   student_id INT PRIMARY KEY,
   name VARCHAR(100),
   age INT,
   course VARCHAR(100)
);
```

10.2 ALTER

Add a column:

```
ALTER TABLE students
ADD COLUMN phone VARCHAR(20);
```

Change table structure using DBMS-specific syntax.

10.3 DROP

Delete a table and its structure:

```
DROP TABLE students;
```

Be careful: this removes the table.

10.4 TRUNCATE

Remove all rows while keeping the table structure.

```
TRUNCATE TABLE students;
```

10.5 DROP vs TRUNCATE

DROP	TRUNCATE
Removes table structure	Keeps table structure
Removes table data	Removes table data
Table must be recreated	Table remains available

Exact transactional behavior can vary by DBMS.

Revision Questions

What is DDL?

Write SQL to create a table.

What does ALTER TABLE do?

Differentiate between DROP and TRUNCATE.

Write SQL to add a phone column.

TOPIC 11: SQL DATA MANIPULATION LANGUAGE

11.1 INSERT

```
INSERT INTO students
(student_id, name, age, course)
VALUES
(1, 'Joseph', 20, 'ICT');
```

11.2 UPDATE

```
UPDATE students
SET course = 'Computer Science'
WHERE student_id = 1;
```

11.3 DELETE

```
DELETE FROM students
WHERE student_id = 1;
```

11.4 Important Warning

Never accidentally run:

```
UPDATE students
SET course = 'ICT';
```

if you intended to update only one student.

Without a WHERE clause, all rows may be updated.

Similarly:

DELETE FROM students;

can delete all rows from the table.

11.5 SELECT

SELECT * FROM students;

Select specific columns:

SELECT name, course

FROM students;

Revision Questions

What is DML?

Explain INSERT.

Explain UPDATE.

Explain DELETE.

Write SQL to retrieve student names.

Why is WHERE important?

TOPIC 12: SQL QUERIES AND FUNCTIONS

12.1 WHERE

Used to filter records.

```
SELECT *
FROM students
WHERE age > 20;
```

12.2 AND

```
SELECT *
FROM students
WHERE age > 18
AND course = 'ICT';
```

12.3 OR

```
SELECT *
FROM students
WHERE course = 'ICT'
OR course = 'Business';
```

12.4 ORDER BY

Sort records.

```
SELECT *
FROM students
ORDER BY name ASC;
```

Descending:

```
SELECT *
FROM students
ORDER BY name DESC;
```

12.5 LIKE

Search using patterns.

```
SELECT *
FROM students
WHERE name LIKE 'Jo%';
```

This can match names beginning with Jo.

12.6 Aggregate Functions

COUNT

```
SELECT COUNT(*)
FROM students;
```

SUM

```
SELECT SUM(marks)
FROM students;
```

AVG

```
SELECT AVG(marks)
```

FROM students;

MAX

SELECT MAX(marks)

FROM students;

MIN

SELECT MIN(marks)

FROM students;

12.7 GROUP BY

Groups records.

SELECT course, COUNT(*)

FROM students

GROUP BY course;

12.8 HAVING

Filters groups.

SELECT course, COUNT(*)

FROM students

GROUP BY course

HAVING COUNT(*) > 5;

12.9 DISTINCT

Removes duplicate results.

```
SELECT DISTINCT course
FROM students;
```

12.10 JOINS

Joins combine related records from multiple tables.

INNER JOIN

Returns matching records.

```
SELECT students.name, courses.course_name
FROM students
INNER JOIN courses
ON students.course_id = courses.course_id;
```

LEFT JOIN

Returns all records from the left table and matching records from the right table.

```
SELECT students.name, courses.course_name
FROM students
LEFT JOIN courses
ON students.course_id = courses.course_id;
```

12.11 SQL Query Example

Suppose:

Students

ID	Name	CourseID	Marks
1	Joseph	101	85
2	Joel	102	72
3	Mercy	101	90

Find students with marks greater than 80:

```
SELECT name, marks
FROM students
WHERE marks > 80;
```

Result:

Name	Marks
Joseph	85
Mercy	90

Revision Questions

What is the purpose of WHERE?

Explain ORDER BY.

What does GROUP BY do?

What is HAVING?

What does DISTINCT do?

Explain an INNER JOIN.

Explain a LEFT JOIN.

State five aggregate functions.

TOPIC 13: DATABASE SECURITY, BACKUP AND RECOVERY

13.1 Database Security

Database security involves protecting databases from:

Unauthorized access

Data theft

Data modification

Data destruction

Malware

Accidental deletion

13.2 Authentication

Authentication verifies the identity of a user.

Examples:

Username/password

Multi-factor authentication

Security tokens

13.3 Authorization

Authorization determines what an authenticated user is allowed to do.

Example:

Administrator

↓

Create / Update / Delete

Teacher

↓

View / Add Marks

Student

↓

View Results

13.4 SQL GRANT

Permissions can be granted to users or roles.

Example:

```
GRANT SELECT
ON students
TO student_user;
```

The exact syntax and privilege model may vary by DBMS.

13.5 SQL REVOKE

Permissions can be removed.

```
REVOKE SELECT
ON students
FROM student_user;
```

13.6 Database Backup

A backup is a copy of database data that can be used for recovery.

Types include:

Full Backup

Copies the complete database.

Incremental Backup

Copies changes made since a previous backup.

Differential Backup

Copies changes made since the last full backup.

13.7 Database Recovery

Recovery restores a database after:

Hardware failure

Software failure

Human error

Data corruption

Security incidents

13.8 Transactions

A transaction is a logical unit of database operations.

Example:

Transfer KSh 5,000

↓

Subtract from Account A

↓

Add to Account B

Both operations should succeed together or the transaction should be rolled back.

13.9 ACID Properties

Atomicity

A transaction is treated as a whole.

Consistency

A transaction should leave the database in a valid state.

Isolation

Concurrent transactions should not improperly interfere with one another.

Durability

Committed changes should survive failures according to the DBMS's durability guarantees.

Revision Questions

Define database security.

Differentiate authentication and authorization.

What is a database backup?

Explain full backup.

Explain incremental backup.

What is a transaction?

Explain ACID properties.

What are GRANT and REVOKE used for?

TOPIC 14: DATABASE PROJECTS AND REVISION

PROJECT 1: STUDENT MANAGEMENT DATABASE

A TVET student management system can contain:

Students Table

```
CREATE TABLE students (
   student_id INT PRIMARY KEY,
   name VARCHAR(100) NOT NULL,
   gender VARCHAR(20),
   phone VARCHAR(20),
   course_id INT
);
```

Courses Table

```
CREATE TABLE courses (
```

```
    course_id INT PRIMARY KEY,
    course_name VARCHAR(100) NOT NULL
);
```

Insert Courses

```
INSERT INTO courses
(course_id, course_name)
VALUES
(101, 'ICT'),
(102, 'Computer Science'),
(103, 'Business');
```

Insert Students

```
INSERT INTO students
(student_id, name, gender, phone, course_id)
VALUES
(1, 'Joseph', 'Male', '0720349953', 101),
(2, 'Joel', 'Male', '0712345678', 102),
(3, 'Mercy', 'Female', '0798765432', 101);
```

PROJECT 2: SCHOOL DATABASE

Possible tables:

Students

Teachers

Courses

Departments

Classes

Marks

Attendance

Possible relationships:

Department

```
    |
    ├── Teachers
    |
    └── Courses
```

Student

```
    |
    ├── Marks
    └── Attendance
```

PROJECT 3: LIBRARY DATABASE

Tables:

Books

Members

Authors

Borrowings

Categories

Example:

```
CREATE TABLE books (
   book_id INT PRIMARY KEY,
   title VARCHAR(200),
   author VARCHAR(150),
   category VARCHAR(100),
   quantity INT
);
```

PROJECT 4: BANK DATABASE

Tables:

Customers

Accounts

Transactions

Branches

Possible relationship:

Customer

```
  |
  └── Account
      |
      └── Transactions
```

PROJECT 5: TVET ICT HUB DATABASE

Your own TVET ICT Hub can use a database such as PostgreSQL.

Possible tables:

users

students

courses

subjects

notes

categories

quizzes

questions

answers

results

admins

For example:

Users

```
CREATE TABLE users (
   user_id SERIAL PRIMARY KEY,
   name VARCHAR(100) NOT NULL,
   email VARCHAR(150) UNIQUE NOT NULL,
   password_hash TEXT NOT NULL
);
```

Subjects

```
CREATE TABLE subjects (
   subject_id SERIAL PRIMARY KEY,
   subject_name VARCHAR(100) NOT NULL
);
```

Notes

```
CREATE TABLE notes (
   note_id SERIAL PRIMARY KEY,
   title VARCHAR(200) NOT NULL,
   content TEXT,
   subject_id INT REFERENCES subjects(subject_id)
);
```

This allows your TVET Hub to connect subjects → notes using a foreign key.

DATABASE DESIGN PROCESS

When creating a database project, follow these steps:

Identify Requirements

↓

Identify Entities

↓

Identify Attributes

↓

Identify Primary Keys

↓

Identify Relationships

↓

Create ERD

↓

Normalize Tables

↓

Create Database

↓

Create Tables

↓

Add Constraints

↓

Insert Data

↓

Create Queries

↓

Test Database

↓

Backup Database

DBMS PRACTICAL ACTIVITIES

Practical 1: Create a Database

Students should create a database called:

college

Practical 2: Create a Student Table

Create:

student_id

name

age

course

phone

Use an appropriate primary key.

Practical 3: Insert Data

Insert at least five students.

Practical 4: Retrieve Data

Display:

All students

Students taking ICT

Students above age 20

Students sorted alphabetically

Practical 5: Update Data

Change the course of one student.

Practical 6: Delete Data

Delete a selected student using their ID.

Practical 7: Create Relationships

Create:

Students

Courses

Connect them using a foreign key.

Practical 8: Create an ERD

Design an ERD for a:

College Management System

Entities should include:

Student

Course

Teacher

Department

Marks

DBMS REVISION QUESTIONS

SECTION A: MULTIPLE CHOICE

1. What is a DBMS?

A. Programming language

B. Software used to manage databases

C. Operating system

D. Web browser

Answer: B

2. Which key uniquely identifies a record?

A. Foreign key

B. Primary key

C. Composite key only

D. Alternate key

Answer: B

3. Which SQL command retrieves data?

A. INSERT

B. SELECT

C. DELETE

D. UPDATE

Answer: B

4. Which SQL command creates a table?

A. CREATE TABLE

B. MAKE TABLE

C. NEW TABLE

D. BUILD TABLE

Answer: A

5. Which constraint prevents duplicate values?

A. CHECK

B. UNIQUE

C. DEFAULT

D. NULL

Answer: B

6. Which key creates a relationship between tables?

A. Primary key

B. Foreign key

C. Candidate key

D. Alternate key

Answer: B

7. What does SQL stand for?

A. Structured Query Language

B. Simple Query Language

C. System Query Logic

D. Structured Question Language

Answer: A

8. Which command modifies existing records?

A. UPDATE

B. MODIFY TABLE

C. CHANGE

D. ALTER DATA

Answer: A

9. Which command removes selected records?

A. DROP

B. DELETE

C. REMOVE

D. CLEAR

Answer: B

10. Which function counts records?

A. SUM()

B. TOTAL()

C. COUNT()

D. NUMBER()

Answer: C

SECTION B: SHORT-ANSWER QUESTIONS

Define a database.

Define a DBMS.

State five advantages of a DBMS.

Define a table.

Define a record.

Define a field.

What is a primary key?

What is a foreign key?

Define normalization.

Explain 1NF.

Explain 2NF.

Explain 3NF.

What is an ERD?

Define cardinality.

What is SQL?

State five SQL commands.

What is a JOIN?

Differentiate between INNER JOIN and LEFT JOIN.

What is a transaction?

Explain ACID.

What is database backup?

What is database recovery?

Differentiate authentication and authorization.

What is referential integrity?

What is a composite key?

SECTION C: PRACTICAL EXAM QUESTIONS

Question 1

Create a database called:

college

Create a students table containing:

student_id

name

age

course

marks

Make student_id the primary key.

Question 2

Insert five students into the table.

Question 3

Write SQL statements to:

Display all students.

Display students who scored above 70.

Display students in ICT.

Sort students by marks.

Find the highest mark.

Find the average mark.

Count the number of students.

Update a student's marks.

Delete a student.

Question 4

Create the following tables:

Students

Courses

Create a foreign-key relationship between them.

Question 5

Design an ERD for a TVET College Management System containing:

Students

Courses

Departments

Teachers

Marks

Identify:

Primary keys

Foreign keys

Relationships

Cardinalities

DBMS QUICK GLOSSARY

Term Meaning

Database Organized collection of related data

DBMS Software used to manage databases

Table Collection of rows and columns

Record A row in a table

Field A column/attribute

Entity Something about which data is stored

Attribute Characteristic of an entity

Primary Key Uniquely identifies a record

Foreign Key References a key in another table

Candidate Key Possible unique identifier

Composite Key Key made from multiple columns

ERD Diagram showing entities and relationships

SQL Structured Query Language

DDL Data Definition Language

DML Data Manipulation Language

DQL Data Query Language

DCL Data Control Language

TCL Transaction Control Language

JOIN Combines related data from tables

Normalization Organizing data to reduce redundancy

Transaction	Logical unit of database work
Backup	Copy of data for recovery
Recovery	Restoring data/system after failure
Constraint	Rule applied to database data
Schema	Structure/design of a database

FINAL DBMS REVISION CHECKLIST

Before an examination, a student should be able to:

- Define data
- Define information
- Define database
- Define DBMS
- Explain functions of a DBMS
- Explain database models
- Explain relational databases
- Create tables
- Explain records and fields
- Use data types
- Explain primary keys
- Explain foreign keys
- Use constraints
- Design an ERD
- Explain cardinality

Explain normalization

Apply 1NF

Apply 2NF

Apply 3NF

Write SQL

Create databases

Create tables

Insert records

Update records

Delete records

Retrieve records

Filter records using WHERE

Sort records using ORDER BY

Group records using GROUP BY

Use aggregate functions

Use JOIN

Explain database security

Explain authentication and authorization

Explain database backup and recovery

Explain transactions

Explain ACID properties

Build a complete database project